

- [Monads, Algebraic Theories, and Operads](#)
 - [What Are Monads?](#)
 - [Monad as “Monoid in the Category of Endofunctors”](#)
 - [Two Fundamental Categories of Algebras](#)
 - [Kleisli Category: \$Kl\(T\)\$](#)
 - [Eilenberg-Moore Category: \$EM\(T\) \equiv C^T\$](#)
 - [Lawvere Theories](#)
 - [The Connection](#)
 - [Operations as Morphisms](#)
 - [Operads: Higher Arity Operations](#)
 - [Operad Definition](#)
 - [PROPS: Multi-Input/Multi-Output Operations](#)
 - [Hierarchy: Operads \$\subset\$ PROPs \$\subset\$ Monads](#)
 - [Programming Language Applications](#)
 - [Moggi’s Semantics \(1989\)](#)
 - [Haskell’s Monad Typeclass](#)
 - [Monad Laws in Programming](#)
 - [Free and Forgetful Adjunction](#)
 - [Syntax-Semantics Bridge](#)
 - [Free Algebras: Syntax](#)
 - [Models: Semantics](#)
 - [Limitations: Effects Beyond Monads](#)
 - [Key Theorems](#)
 - [Common Mistakes](#)
 - [Study Guide](#)
 - [Further Reading](#)
 - [Key Questions](#)

Monads, Algebraic Theories, and Operads

What Are Monads?

A **monad** on a category C is a triple $T = (T, \eta, \mu)$ where: - $T: C \rightarrow C$ is an endofunctor - $\eta: id_C \Rightarrow T$ is the unit (natural transformation) - $\mu: T^2 \Rightarrow T$ is the multiplication (natural transformation)

Coherence Laws:

Associativity:

$$\begin{array}{ccc}
 T^3 & \xrightarrow{\mu T} & T^2 \xrightarrow{\mu} T \\
 | & & | \\
 T\mu & & | \\
 | & & | \\
 T^2 & \xrightarrow{\mu} & T
 \end{array}$$

Left Unit: $id \cdot T = T$ (composition with η)

Right Unit: $T \cdot id = T$ (composition with η)

Monad as “Monoid in the Category of Endofunctors”

Just as a monoid $(M, \cdot, e)$ has multiplication and identity in any monoid category, a monad has “multiplication” μ and “identity” η in the monoidal category of endofunctors $[C, C]$.

Key insight: Monads abstract algebraic structure independently of presentations.

Two Fundamental Categories of Algebras

Kleisli Category: $Kl(T)$

Objects: Elements of C (same objects as C) **Morphisms:** $f: X \multimap Y$ in $Kl(T)$ are morphisms $f: X \rightarrow T(Y)$ in C

Composition:

$$(X \multimap Y) \circ (Y \multimap Z) = (X \multimap T(Y) \rightarrow T(Z)) = T(Z)$$

Universal property: $Kl(T)$ is the initial T -algebra category. Free T -algebras are objects with unique structure.

Eilenberg-Moore Category: $EM(T) = C^T$

Objects: T -algebras (X, h) where $h: T(X) \rightarrow X$ respects structure - $h \circ \eta_X = id_X$ (left unit) - $h \circ \mu_X = h \circ T(h)$ (associativity)

Morphisms: $f: (X, h) \rightarrow (Y, k)$ where $k \circ T(f) = f \circ h$ (f respects structure)

Universal property: $EM(T)$ is the terminal T -algebra category. All T -algebras live here with all natural transformations.

Lawvere Theories

The Connection

Theorem (Lawvere): Finitary monads T on $Set \leftrightarrow$ Lawvere theories L

What is a Lawvere theory? - Small category L with finite products - Full embedding $Fin^{op} \rightarrow L$ (where $Fin =$ finite sets) - Models: product-preserving functors $L \rightarrow Set$

Examples: - Monoid theory: Operations $\{e, m\}$ with axioms - Group theory: $\{e, m, i\}$ with equations - Ring theory: $\{+, 0, \cdot, 1\}$ with bilinear relations

Operations as Morphisms

In Lawvere theory for groups:

n -ary operation: $T_n \rightarrow T_1$
(n copies of T_1 to single copy via composition rules)

Equations become commutative diagrams:

Associativity: $T_3 \rightarrow T_1$
 $\downarrow$

Commutativity diagrams encode equations

Operads: Higher Arity Operations

Operad Definition

An **operad** in a monoidal category $(V, \otimes)$ consists of: - **O(n)** for each $n \geq 0$ (space of n-ary operations) -

Composition maps: $\gamma_{\{k; n_1, \dots, n_k\}}: O(k) \otimes O(n_1) \otimes \dots \otimes O(n_k) \rightarrow O(n_1 + \dots + n_k)$ - **Unit element** in $O(1)$ - **Associativity and unit axioms**

PROPS: Multi-Input/Multi-Output Operations

PROP (Product and Permutation category) generalizes operads: - Operations have multiple inputs AND multiple outputs - Example: Matrix operations, tangle diagrams

Hierarchy: Operads $\subset$ PROPs $\subset$ Monads

Every operad generates a monad (free algebras) Every PROP generates multiple monads (one per output sort)

Programming Language Applications

Moggi's Semantics (1989)

Insight: **Computational effects = monads**

Values vs Computations: - **Values:** Elements of C - **Computations:** Elements of $T(C)$ (what computation returns)

Haskell's Monad Typeclass

```
class Monad m where
    return :: a -> m a           -- unit  $\eta$ 
    (>=>) :: m a -> (a -> m b) -> m b -- bind (Kleisli composition)
```

Examples: - **Maybe:** Exceptions (return value or nothing) - **State s:** Mutable state threading - **List:** Nondeterminism (multiple results) - **IO:** Side effects (input/output actions) - **Reader r:** Environment/dependency injection

Monad Laws in Programming

```
-- Left identity:  return a >=> f  $\equiv$  f a
-- Right identity: m >=> return  $\equiv$  m
-- Associativity:  (m >=> f) >=> g  $\equiv$  m >=> (x -> f x >=> g)
```

These are η , η , μ coherence laws!

Free and Forgetful Adjunction

Every monad arises from an adjunction:

Theorem: T monad on $C \implies \exists F \dashv U$ (free-forgetful)

Construction: - **U:** $C^T \rightarrow C$ (forget algebra structure) - **F:** $C \rightarrow C^T$ (free algebra on element) - **T = U $\circ$ F** (the induced monad)

Converse: Given $F \dashv U$, define $T = U \circ F$ monad.

Syntax-Semantics Bridge

Free Algebras: Syntax

For monad T on C : - $T(X)$ = “terms with variables from X ” - No particular interpretation (purely syntactic) - Think: formal expressions, abstract syntax trees

Models: Semantics

- T -algebras $h: T(X) \rightarrow X$ (interpretation)
- Each algebra structure gives different meaning
- Think: evaluation functions, semantic domains

Yoneda Extension: Universal property ensures variable assignments extend uniquely to homomorphisms:

$$\begin{array}{ccc} X & \rightarrow & T(X) \text{ [variables in syntax]} \\ \downarrow & & \downarrow \\ Y & \rightarrow & h(Y) \text{ [through any algebra } h] \end{array}$$

Limitations: Effects Beyond Monads

Not all computational effects are monadic:

Effect	Monad?	Alternative
Exceptions	✓	Monad
State	✓	Monad
Nondeterminism	✓	Monad
Continuations	✗	Not monad (contravariant)
Probabilistic	✗	Indexed monads
Concurrent	✗	Applicative/Alternative
Algebraic effects ~		Effect handlers

Key Theorems

Theorem	Content
Eilenberg-Moore	T -algebras form complete abelian category
Kleisli	Free category for T
Lawvere	Finitary monads $\leftrightarrow$ equational theories
Monadicity	When is functor G monadic?

Common Mistakes

✗ **Mistake:** “Monad = effect” ✓ **Reality:** Monads model certain effects (many don’t fit monad structure)

✗ **Mistake:** “Kleisli = simplest T -algebras” ✓ **Reality:** Kleisli has *free* algebras, EM has *all* algebras

✗ **Mistake:** “Operads = monads with multiple outputs” ✓ **Reality:** Operads are different (symmetric operations, different composition)

✗ **Mistake:** “Lawvere theories are outdated” ✓ **Reality:** Fundamental tool for logic and semantics (still highly relevant)

Study Guide

1. Understand unit-counit laws deeply
2. Work through Maybe monad in detail
3. Prove Kleisli category is a category
4. Understand free-forgetful connection
5. Apply to programming effects
6. Connect to Lawvere theories
7. Recognize operads in your field

Further Reading

- Moggi, “Notions of Computation and Monads” (1989 seminal paper)
- Mac Lane, “Categories for the Working Mathematician” (Chapter VI)
- Awodey, “Category Theory” (Chapter 10)
- Kleisli and Roiter, “Abelian Categories”
- Markl, Shnider, Stasheff, “Operads in Algebra, Topology, and Physics”

Key Questions

1. Why is the monad just the unit and multiplication?
2. How do Kleisli and EM categories relate?
3. Why are Lawvere theories equivalent to monads?
4. How do operads generalize monads?
5. What effects can NOT be expressed as monads?
6. Why is syntax-semantics separation important?